Práctica 1 — Par de puntos más cercano

Autor: Acoidan Martín Conrado

Asignatura: Algorítmica y Modelos de Computación

Implementación: Angular y TypeScript

Curso de referencia: 2024–2025

Resumen

Esta práctica estudia el problema geométrico del par de puntos más cercano: dado un conjunto de n puntos del plano, encontrar los dos cuya distancia euclídea sea mínima. Se implementan y comparan cuatro estrategias: búsqueda exhaustiva, búsqueda exhaustiva con poda, divide y vencerás básico y divide y vencerás mejorado.

La aplicación permite cargar o generar datasets, representar gráficamente la solución, comprobar que las cuatro estrategias obtienen la misma distancia y ejecutar un estudio experimental reproducible. Las mediciones se realizan fuera del hilo principal mediante un Web Worker y excluyen la generación de datos, la representación gráfica y la preparación de copias.

Los resultados muestran que no existe una estrategia ganadora para todas las distribuciones. En el caso general estudiado, la búsqueda exhaustiva con poda obtiene el menor tiempo gracias a su sencillez y a una poda muy efectiva. Cuando todos los puntos comparten coordenada x, las tres estrategias no mejoradas calculan todos los pares y divide y vencerás mejorado conserva un crecimiento claramente inferior.

Requisitos y trazabilidad

La tabla relaciona los requisitos principales del enunciado con su implementación y evidencia.

Requisito	Implementación	Evidencia
Cargar datasets TSPLIB	`tsplib-parser.ts` e interfaz de archivos	Pruebas válidas e inválidas del parser y del componente

Requisito	Implementación	Evidencia
Generar datasets general y vertical	`dataset-generator.ts` y `tsplib-writer.ts`	Descarga TSPLIB y pruebas de formato
Implementar HeapSort	`heap-sort.ts`	Orden por x e y, sin `Array.sort()` en producción
Ejecutar cuatro estrategias	Módulos de búsqueda y `strategy-runner.ts`	Igualdad de distancias y contadores independientes
Mostrar solución textual y gráfica	Tablas y `point-plot.ts`	Pareja mínima, distancia, puntos y segmento resaltado
Comparar tallas 200, 500, 1500 y 5000	`experiment-runner.ts`	Diez repeticiones por talla y progreso de 40 experimentos
Mantener la interfaz activa	`experiment.worker.ts`	Cálculo en Worker con cancelación y control de errores
Presentar tabla y gráfica conjunta	`experiment-chart.ts` y CSV	Cuatro líneas, tablas y ficheros de evidencia
Entregar documentación razonada	VitePress, esta memoria y TypeDoc	Portal web imprimible a PDF y referencia técnica

---

Definición formal del problema

Sea un conjunto de puntos del plano:

$$P = \{p_1, p_2, \ldots, p_n\}, \qquad p_i = (x_i, y_i).$$

Para dos puntos $p = (p_x, p_y)$ y $q = (q_x, q_y)$ se utiliza la distancia euclídea:

$$d(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}.$$

El objetivo consiste en obtener:

$$(p^*, q^*) = \arg\min_{p \neq q} d(p, q).$$

Si existen varias parejas empatadas, cualquiera con la distancia mínima es una solución válida. Por ese motivo, las comprobaciones automáticas comparan las distancias obtenidas y no exigen que coincidan los identificadores de la pareja.

Modelo de datos e instrumentación

Cada punto conserva un identificador y sus dos coordenadas. Sus propiedades son inmutables; las estrategias que ordenan modifican únicamente la posición de las referencias dentro de su array de trabajo.

Todas las estrategias reciben un contador propio. La única función que lo incrementa es la operación central de distancia:

```
distanceBetween(p, q, contador):
    deltaX = p.x - q.x
    deltaY = p.y - q.y
    distancia = raíz(deltaX² + deltaY²)
    contador = contador + 1
    devolver distancia
```

Esta instrumentación permite contar exactamente cuántas distancias euclídeas calcula cada algoritmo. Las restas de coordenadas empleadas como cotas no incrementan el contador.

La métrica instrumental no representa todo el tiempo de ejecución. Ordenar, crear arrays, consultar mapas, particionar y gestionar la recursión también tiene coste. Por ello se estudian conjuntamente:

- tiempo transcurrido;
- número de distancias euclídeas;
- complejidad asintótica del algoritmo completo.

Política de copias y frontera temporal

Cada estrategia recibe una copia superficial independiente del array original. Los objetos `Point` pueden compartirse porque son inmutables, pero el orden de los arrays no se comparte.

```
Dataset original
├── copia para Exhaustivo
├── copia para Exhaustivo con Poda
├── copia para Divide y Vencerás
└── copia para Divide y Vencerás mejorado
```

La copia se prepara antes de iniciar el reloj. La ordenación sí forma parte de la estrategia y queda incluida en la medición.

HeapSort

Diseño

Los puntos se ordenan lexicográficamente. Para el orden por x se utiliza:

$$x \to y \to id,$$

y para el orden por y:

$$y \to x \to id.$$

El último desempate garantiza un orden total determinista. HeapSort representa un árbol binario completo dentro del propio array. Para un nodo situado en i:

$$hijoIzquierdo(i) = 2i + 1, \qquad hijoDerecho(i) = 2i + 2.$$

Pseudocódigo

```
```

```
HEAPSORT(A, comparar):
    para raíz desde floor(|A| / 2) - 1 hasta 0
        BAJAR(A, raíz, |A|, comparar)

    para fin desde |A| - 1 hasta 1
        intercambiar A[0] y A[fin]
        BAJAR(A, 0, fin, comparar)

BAJAR(A, raíz, tamañoHeap, comparar):
    actual = raíz

    repetir
        mayor = actual
        izquierdo = 2·actual + 1
        derecho = 2·actual + 2

        si izquierdo pertenece al heap y A[izquierdo] > A[mayor]
            mayor = izquierdo

        si derecho pertenece al heap y A[derecho] > A[mayor]
            mayor = derecho

        si mayor = actual
            terminar

        intercambiar A[actual] y A[mayor]
        actual = mayor
```

Correctitud y complejidad

La fase de construcción aplica `BAJAR` desde el último padre hasta la raíz. Como los hijos de cada nodo ya forman montículos, cada llamada restaura la propiedad de max-heap en su subárbol. Después de construirlo, el máximo global está en la raíz.

Cada extracción intercambia la raíz con la última posición activa, reduce el heap y restaura su propiedad. Los máximos quedan fijados de derecha a izquierda, por lo que el array final queda ordenado de forma ascendente.

Construir el montículo cuesta $\Theta(n)$. Se realizan $n-1$ extracciones, cada una con coste $O(\log n)$:

$$T(n) = \Theta(n \log n).$$

El coste es $\Theta(n \log n)$ en los casos favorable, medio y desfavorable. La ordenación es *in-place* y utiliza $O(1)$ espacio auxiliar.

Estrategia 1 — Búsqueda exhaustiva

Idea y pseudocódigo

La estrategia recorre una única vez cada pareja no ordenada.

```
EXHAUSTIVO(P):
    mejor = indefinido

    para i desde 0 hasta |P| - 2
        para j desde i + 1 hasta |P| - 1
            distancia = DISTANCIA(P[i], P[j])

            si mejor no existe o distancia < mejor.distancia
                mejor = (P[i], P[j], distancia)

    devolver mejor
```

Correctitud

Toda pareja de índices distintos puede escribirse de forma única como (i, j) con $i < j$. Los dos bucles recorren exactamente esas combinaciones. Después de procesar cada pareja, `mejor` conserva la menor distancia vista hasta ese momento. Al terminar, se han examinado todas las soluciones posibles y el invariante garantiza que `mejor` es globalmente óptima.

Complejidad

El número exacto de distancias es:

$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

Por tanto:

$$T(n) = \Theta(n^2)$$

para los casos favorable, medio y desfavorable. La distribución geométrica no altera el recorrido. El espacio auxiliar es $O(1)$, sin contar la copia de entrada realizada por el coordinador.

Estrategia 2 — Búsqueda exhaustiva con poda

Cota horizontal

Para dos puntos cualesquiera:

$$d(p_i, p_j) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2} \geq |x_j - x_i|.$$

Después de ordenar por x, fijado i, la diferencia $x_j - x_i$ no disminuye al aumentar j. Si esa diferencia ya es mayor o igual que la mejor distancia conocida, ni el punto actual ni los posteriores pueden mejorarla.

Pseudocódigo

```
EXHAUSTIVO-CON-PODA(P):
    HEAPSORT-POR-X(P)
    mejor = indefinido

    para i desde 0 hasta |P| - 2
        para j desde i + 1 hasta |P| - 1
            deltaX = P[j].x - P[i].x

            si mejor existe y deltaX >= mejor.distancia
                abandonar el bucle interior

            distancia = DISTANCIA(P[i], P[j])

            si mejor no existe o distancia < mejor.distancia
                mejor = (P[i], P[j], distancia)

    devolver mejor
```

La condición se comprueba antes de calcular la distancia. Se utiliza `>=` porque una pareja empatada

no necesita sustituir la primera solución encontrada.

Correctitud

La ordenación no elimina puntos, solo cambia su posición. Antes de la poda, las parejas se recorren igual que en el exhaustivo. Cuando se ejecuta el corte se cumple:

$$d(P_i, P_j) \geq x_j - x_i \geq d_{min}.$$

Para cualquier punto posterior $k > j$, $x_k - x_i \geq x_j - x_i$, por lo que tampoco puede mejorar la solución. Así, únicamente se descartan candidatos que no pueden ser estrictamente mejores.

Complejidad

- **Caso favorable:** una distancia pequeña permite cortar casi inmediatamente cada bucle. La búsqueda es lineal o cercana a lineal y HeapSort domina: $\Theta(n \log n)$.
- **Caso medio esperado:** con puntos razonablemente dispersos, la poda suele mantener pocos vecinos horizontales y el comportamiento esperado queda dominado por la ordenación: $\Theta(n \log n)$.
- **Caso desfavorable:** si todos los puntos tienen la misma x, entonces $\Delta x = 0$ y la poda no actúa. Se calculan todos los pares: $\Theta(n^2)$.

HeapSort trabaja *in-place*, por lo que el algoritmo utiliza $O(1)$ espacio auxiliar, sin contar la copia externa de entrada.

Estrategia 3 — Divide y vencerás básico

División del espacio de soluciones

Tras ordenar por x, toda pareja pertenece exactamente a uno de tres grupos:

1. ambos puntos están en la mitad izquierda;
2. ambos están en la mitad derecha;
3. un punto está en cada mitad.

Las llamadas recursivas resuelven los dos primeros grupos. La combinación examina las parejas

cruzadas que aún podrían mejorar el mínimo interno.

Pseudocódigo

```
DIVIDE-Y-VENCERÁS-BÁSICO(P):
    HEAPSORT-POR-X(P)
    devolver RESOLVER(P, 0, |P|)

RESOLVER(P, inicio, fin):
    si fin - inicio <= 3
        devolver EXHAUSTIVO-DEL-RANGO(P, inicio, fin)

    medio = inicio + floor((fin - inicio) / 2)
    mejorIzquierda = RESOLVER(P, inicio, medio)
    mejorDerecha = RESOLVER(P, medio, fin)
    mejor = mínimo(mejorIzquierda, mejorDerecha)

    xDivisoria = P[medio].x
    franjaIzquierda = puntos de [inicio, medio) a menos de mejor.distancia
    franjaDerecha = puntos de [medio, fin) a menos de mejor.distancia

    para cada p en franjaIzquierda
        para cada q en franjaDerecha
            si q.x - p.x >= mejor.distancia
                abandonar el bucle interior

            distancia = DISTANCIA(p, q)
            actualizar mejor si distancia es menor

    devolver mejor
```

Las mitades se definen mediante índices y no únicamente por el valor de x. Esto evita ambigüedades cuando varios puntos comparten coordenada horizontal.

Correctitud

Por inducción sobre el tamaño del rango:

- los rangos de hasta tres puntos se resuelven exhaustivamente;
- por hipótesis inductiva, cada llamada devuelve el mínimo correcto de su mitad;
- una pareja cruzada que mejore $\delta = \min(d_{izq}, d_{der})$ debe tener ambos puntos a menos de δ de la

divisoria;

- la combinación examina esas parejas, y la poda horizontal solo descarta las que ya no pueden mejorar el mínimo.

Los tres grupos cubren todas las parejas, por lo que el resultado es globalmente correcto.

Complejidad

La recurrencia general es:

$$T(n) = 2T(n/2) + C(n).$$

Con una franja pequeña, $C(n) = \Theta(n)$ y:

$$T(n) = \Theta(n \log n).$$

Sin embargo, la franja no está ordenada por y ni existe un límite constante de candidatos. En el caso vertical puede contener todos los puntos y la combinación puede comparar cuadráticamente:

$$T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2).$$

El caso favorable y el medio esperado bajo una distribución general son $\Theta(n \log n)$; el peor caso es $\Theta(n^2)$. Las franjas requieren $O(n)$ espacio auxiliar máximo, además de una pila de recursión $O(\log n)$.

Estrategia 4 — Divide y vencerás mejorado

Dos órdenes simultáneos

La mejora conserva:

- un array ordenado por x para dividir;
- una vista con las mismas referencias ordenada por y para combinar.

Un mapa relaciona cada referencia `Point` con su índice en el orden por x. En cada llamada, la lista ordenada por y se reparte de forma estable entre las mitades. Como el recorrido original ya está ordenado, ambas sublistas mantienen ese orden sin volver a ejecutar HeapSort.

Límite geométrico de once sucesores

Tras resolver ambas mitades se conoce:

$$\delta = \min(d_{izq}, d_{der}).$$

Dentro de cada mitad no existe una pareja a distancia menor que δ. Una celda cuadrada de lado $\delta/2$ tiene diagonal:

$$\sqrt{(\delta/2)^2 + (\delta/2)^2} = \frac{\delta}{\sqrt{2}} < \delta.$$

Por tanto, dos puntos de la misma mitad no pueden ocupar una misma celda. Para un punto, la ventana de candidatos puede cubrir tres filas y cuatro columnas de esas celdas: doce celdas en total. Una contiene el punto actual, por lo que basta examinar como máximo once sucesores en orden por y.

Pseudocódigo

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
```

```
25
26  DIVIDE-Y-VENCERÁS-MEJORADO(P):
27      HEAPSORT-POR-X(P)
28      índiceX = mapa de punto a posición en P
29      PY = copia de P
30      HEAPSORT-POR-Y(PY)
31      devolver RESOLVER-MEJORADO(P, PY, 0, |P|, índiceX)
32
33  RESOLVER-MEJORADO(PX, PY, inicio, fin, índiceX):
34      si fin - inicio <= 3
            devolver EXHAUSTIVO-DEL-RANGO(PX, inicio, fin)

        medio = inicio + floor((fin - inicio) / 2)
        repartir PY establemente en izquierdaY y derechaY según índiceX

        mejorIzquierda = RESOLVER-MEJORADO(PX, izquierdaY, inicio, medio, índiceX)
        mejorDerecha = RESOLVER-MEJORADO(PX, derechaY, medio, fin, índiceX)
        mejor = mínimo(mejorIzquierda, mejorDerecha)

        franja = puntos de PY con |p.x - PX[medio].x| < mejor.distancia

        para i desde 0 hasta |franja| - 2
            último = min(i + 11, |franja| - 1)

            para j desde i + 1 hasta último
                si franja[j].y - franja[i].y >= mejor.distancia
                    abandonar el bucle interior

                si ambos puntos pertenecen a la misma mitad
                    continuar

                distancia = DISTANCIA(franja[i], franja[j])
                actualizar mejor si distancia es menor

        devolver mejor
```

Correctitud

La división cubre las parejas internas y cruzadas igual que la versión básica. La partición por referencias conserva exactamente la pertenencia a las mitades incluso cuando hay empates de x. La franja mantiene el orden por y.

Las parejas del mismo lado ya fueron resueltas recursivamente. Para las cruzadas, el argumento

geométrico garantiza que una pareja capaz de mejorar δ aparece entre los once sucesores considerados. La poda vertical es segura porque, al estar ordenada la franja, las diferencias $y_j - y_i$ posteriores no disminuyen.

Complejidad

Las dos ordenaciones iniciales cuestan $\Theta(n \log n)$. En cada nivel:

- la partición estable cuesta $\Theta(n)$;
- construir la franja cuesta $\Theta(n)$;
- comprobar como máximo once sucesores por punto cuesta $\Theta(n)$.

La recurrencia es:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

El algoritmo mantiene $\Theta(n \log n)$ en los casos favorable, medio y desfavorable. Las vistas, el mapa y las particiones activas ocupan $O(n)$ espacio auxiliar; la pila recursiva tiene profundidad $O(\log n)$.

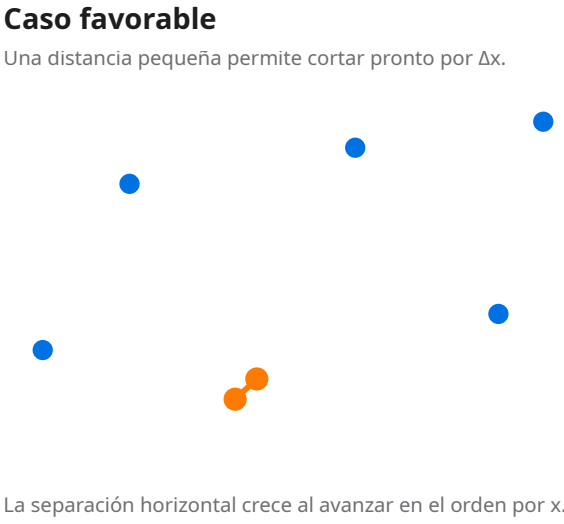
Si se ordenara cada franja desde cero, la recurrencia incluiría $\Theta(n \log n)$ por nivel y el resultado sería $\Theta(n \log^2 n)$. Mantener el orden por y evita esa degradación.

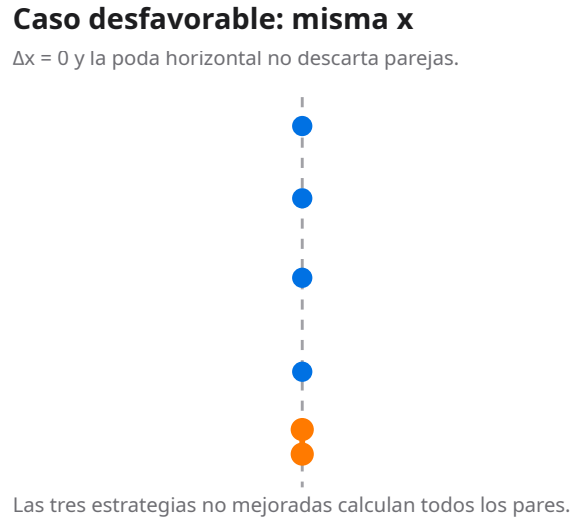
Comparación teórica

Algoritmo	Favorable	Medio esperado	Desfavorable	Espacio auxiliar
HeapSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$
Exhaustivo	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$
Exhaustivo con poda	$\Theta(n \log n)$	$\Theta(n \log n)$ bajo distribución general	$\Theta(n^2)$	$O(1)$
DyV básico	$\Theta(n \log n)$	$\Theta(n \log n)$ bajo distribución general	$\Theta(n^2)$	$O(n)$
DyV mejorado	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$

Los casos medios de las dos estrategias con poda dependiente de la distribución se expresan bajo el modelo experimental de puntos generales. Sin una distribución probabilística concreta, su garantía asintótica universal sigue siendo el peor caso cuadrático.

Ejemplos límite





En el caso favorable aparece pronto una pareja cercana y la separación horizontal crece al avanzar. Esto hace que la búsqueda con poda y la combinación básica descarten gran parte de los candidatos.

En el caso vertical se cumple $\Delta x = 0$ para cualquier pareja. El exhaustivo con poda no puede cortar y la franja de divide y vencerás básico contiene todos los puntos. Divide y vencerás mejorado no depende exclusivamente de esa separación: utiliza el orden por y y un número constante de sucesores.

Arquitectura de la solución

La aplicación separa el núcleo puro de los detalles del navegador:

```
1
2
3
4
```

```
Interfaz Angular
├── carga, formularios, tablas y SVG
├── navegación y exportaciones
└── protocolo de mensajes
    └── Web Worker
        └── motor experimental
            ├── generador de datasets
            ├── medición adaptativa
            └── estrategias
                ├── Exhaustivo
                ├── Exhaustivo con poda
                ├── DyV básico
                └── DyV mejorado
```

Los módulos principales son:

- `core/` : algoritmos, modelos, parser, serialización, medición y transformaciones puras;
- `browser/` : descargas y Web Worker;
- `app.ts` y `app.html` : estado y presentación Angular;
- `results/` : CSV conservados como evidencia;
- `docs/` : memoria VitePress y referencia TypeDoc.

Concurrencia

Una función `async` no crea otro hilo. El estudio completo se ejecuta en un Web Worker para que Angular conserve el control de eventos, progreso y cancelación. La comunicación utiliza datos clonables y mensajes discriminados:

- `start` ;
- `progress` ;
- `completed` ;
- `error` .

El Worker termina al completar, fallar, cancelarse, superar el tiempo de inactividad o destruirse el componente.

Metodología experimental

Configuración

Se estudian las tallas obligatorias:

$$200,\ 500,\ 1500,\ 5000.$$

Para cada talla se realizan diez repeticiones. Dentro de una repetición se genera un único dataset y todas las estrategias reciben copias independientes de esos mismos puntos.

Se analizan dos distribuciones:

- **general:** puntos aleatorios en el plano;
- **misma x:** caso vertical desfavorable para la poda horizontal.

Medición adaptativa

Las ejecuciones pequeñas pueden durar menos que la resolución efectiva del reloj. Para evitar tiempos nulos o excesivamente cuantizados se sigue este procedimiento:

1. una ejecución de calentamiento excluida;
2. una ejecución de calibración excluida;
3. elección de un lote entre 1 y 100 ejecuciones para aproximarse a 5 ms;
4. preparación de todas las copias fuera del reloj;
5. medición conjunta del lote;
6. división del tiempo y del contador entre el número de ejecuciones.

Para una estrategia E, talla n y $r = 10$ repeticiones:

$$\bar{t}_{E,n} = \frac{1}{r} \sum_{k=1}^{r} t_{E,n,k}.$$

Quedan fuera del intervalo medido:

- generación del dataset;
- preparación de copias;
- mensajes de progreso;

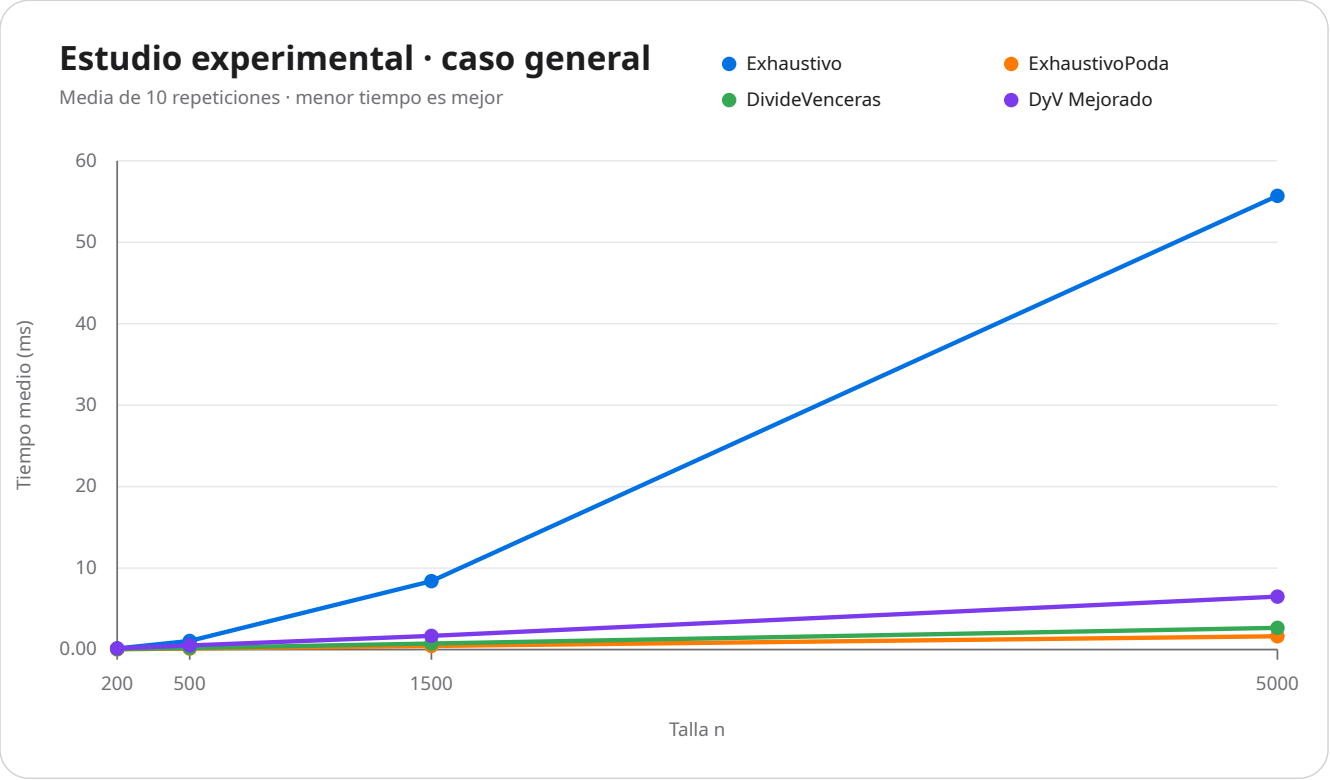
- tablas, gráfica y exportación.

La ordenación requerida por cada estrategia permanece dentro del intervalo.

Resultados experimentales

Caso general

Talla	Exhaustivo	ExhaustivoPoda	DyV básico	DyV mejorado
200	0.134 ms	0.032 ms	0.057 ms	0.141 ms
500	1.060 ms	0.128 ms	0.196 ms	0.500 ms
1500	8.400 ms	0.440 ms	0.740 ms	1.680 ms
5000	55.700 ms	1.633 ms	2.667 ms	6.500 ms



Distancias euclídeas medias:

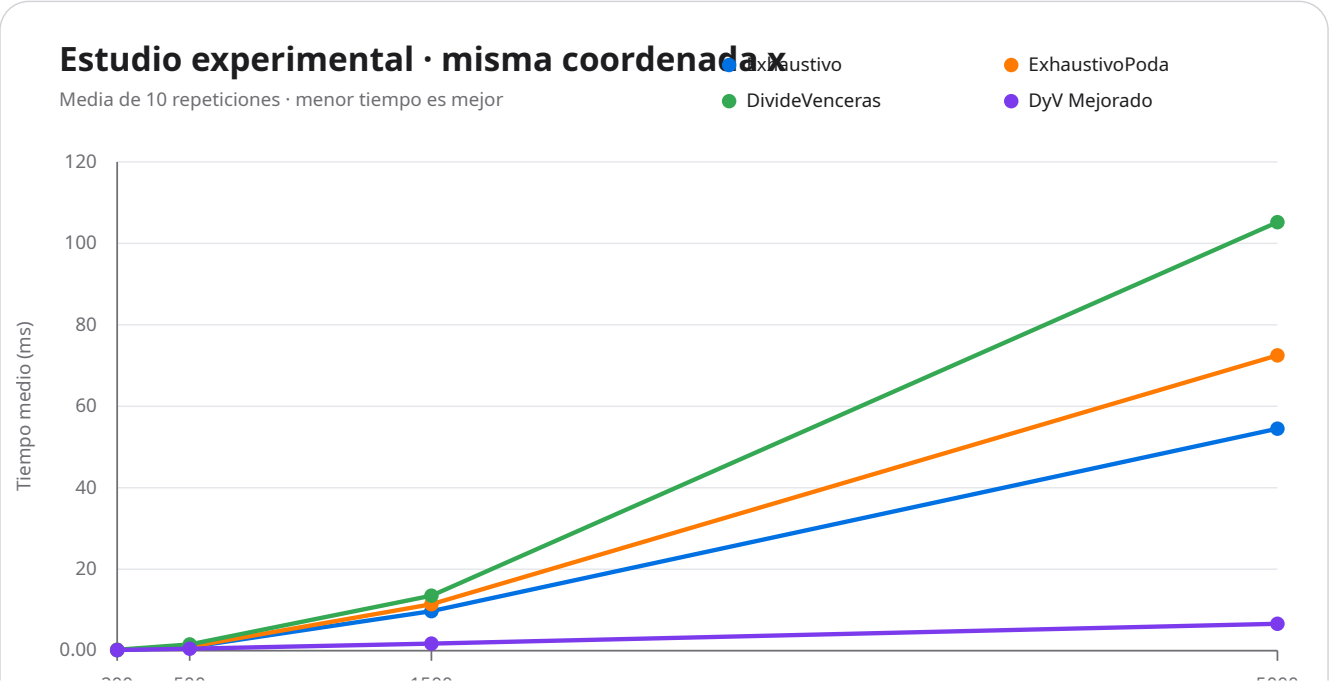
Talla	Exhaustivo	ExhaustivoPoda	DyV básico	DyV mejorado

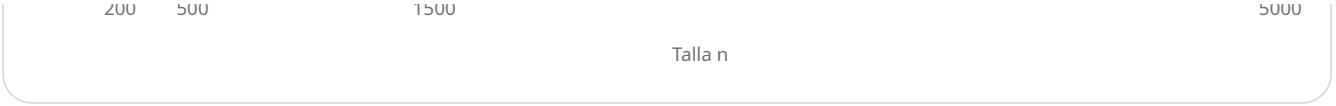
Talla	Exhaustivo	ExhaustivoPoda	DyV básico	DyV mejorado
200	19 900	297.8	685.8	236.8
500	124 750	535.3	2204.8	453.7
1500	1 124 250	2275.1	9812.2	1820.7
5000	12 497 500	7084.0	49 489.4	5355.2

En esta distribución, ExhaustivoPoda es la estrategia más rápida. Para $n = 5000$ tarda aproximadamente 34 veces menos que el exhaustivo. DyV mejorado calcula menos distancias que ExhaustivoPoda, pero su mantenimiento de dos órdenes, mapas, particiones y recursión introduce más trabajo auxiliar.

Caso vertical: misma coordenada x

Talla	Exhaustivo	ExhaustivoPoda	DyV básico	DyV mejorado
200	0.108 ms	0.174 ms	0.220 ms	0.165 ms
500	0.880 ms	0.963 ms	1.533 ms	0.480 ms
1500	9.700 ms	11.400 ms	13.500 ms	1.733 ms
5000	54.500 ms	72.500 ms	105.200 ms	6.600 ms





Distancias euclídeas medias:

Talla	Exhaustivo	ExhaustivoPoda	DyV básico	DyV mejorado
200	19 900	19 900	19 900	194.3
500	124 750	124 750	124 750	317.6
1500	1 124 250	1 124 250	1 124 250	1532.5
5000	12 497 500	12 497 500	12 497 500	4187.4

Para $n = 5000$, las tres estrategias no mejoradas calculan exactamente:

$$\frac{5000 \cdot 4999}{2} = 12\,497\,500$$

distancias. DyV mejorado realiza una media de 4187.4 y obtiene el menor tiempo con 6.6 ms. La diferencia aumenta con la talla, de acuerdo con la separación teórica entre crecimiento cuadrático y $n \log n$.

Evidencia reproducible

Los datos completos pueden descargarse directamente:

- CSV del caso general;
- CSV del caso vertical.

Cada fila conserva distribución, repeticiones, talla, estrategia, método de medición, tamaño del lote, tiempo medio y distancias medias.

Comparación entre teoría y experimento

Los resultados son coherentes con el análisis:

1. **Exhaustivo mantiene crecimiento cuadrático.** Su contador coincide exactamente con $n(n-1)/2$ en ambas distribuciones.

2. **La poda depende de la geometría.** En el caso general reduce millones de distancias a unos pocos miles; con la misma x no elimina ninguna.
3. **DyV básico no garantiza $n \log n$.** Su franja cruzada se degrada en vertical y termina calculando todos los pares.
4. **DyV mejorado es robusto.** Mantiene pocos candidatos incluso cuando falla la cota horizontal.
5. **El orden asintótico no decide todas las tallas.** Para entradas pequeñas, constantes y estructuras auxiliares pueden hacer más lenta una estrategia teóricamente mejor.
6. **El contador no equivale al tiempo.** En el caso general, DyV mejorado calcula menos distancias que ExhaustivoPoda pero tarda más por su sobrecoste estructural.

La conclusión no es que una estrategia sea siempre superior, sino que cada una cambia trabajo geométrico por costes auxiliares distintos.

Pruebas y verificación

La suite automatizada contiene **20 archivos y 103 pruebas**. Cubre, entre otros aspectos:

- distancia e incremento exacto del contador;
- comparadores y HeapSort por x e y;
- arrays vacíos, unitarios y coordenadas repetidas;
- coincidencia de cada estrategia con el exhaustivo;
- mínimos izquierdos, derechos y cruzados;
- cientos de datasets deterministas pequeños;
- peor caso vertical;
- aislamiento de copias y relojes;
- parser y escritor TSPLIB;
- generación de datasets;
- protocolo y cancelación del Worker;
- cálculo de medias, gráfica y CSV;
- navegación accesible y modo oscuro;
- control de inactividad del estudio experimental.

La entrega se comprueba con:

```
npm test -- --watch=false
npm run site:build
npm audit
git diff --check
```

El build de producción genera la aplicación, el Worker, esta documentación y la referencia TypeDoc sin dependencias de servidor.

Despliegue

El proyecto es una aplicación estática: Angular, los algoritmos y el Web Worker se ejecutan en el navegador. Al importar el monorepositorio, Vercel debe usar como **Root Directory** `projects/Algoritmica y Modelos de Computacion/Practica 1`. Después solo necesita instalar dependencias, ejecutar `npm run site:build` y publicar:

```
dist/practica-1-amc/browser
```

`vercel.json` conserva esa configuración. El despliegue contiene:

- aplicación principal en `/`;
- memoria y guía en `/documentacion/`;
- referencia técnica en `/documentacion/api/`.

No existe backend ni se transmiten datasets al servidor.

Limitaciones y amenazas a la validez

- Los tiempos dependen del navegador, CPU, carga del sistema, JIT y resolución del reloj.
- Los datasets generales se generan aleatoriamente; las medias reducen ruido, pero una ejecución posterior no reproduce las mismas coordenadas exactas.
- El lote adaptativo puede variar entre estrategia y talla; el CSV conserva su valor para interpretar la medición.

- La igualdad directa de distancias es adecuada aquí porque todas las estrategias calculan las mismas operaciones sobre las mismas coordenadas; en otros contextos numéricos podría requerirse una tolerancia.
- Los resultados prueban las implementaciones y distribuciones estudiadas, no sustituyen las demostraciones asintóticas.

Estas limitaciones no invalidan la comparación: dentro de cada repetición todas las estrategias reciben el mismo dataset y se ejecutan bajo la misma metodología.

Conclusiones

La práctica conecta tres niveles de análisis:

- **correctitud:** todas las estrategias devuelven la misma distancia mínima;
- **teoría:** las podas y el mantenimiento del orden por y explican los órdenes asintóticos;
- **experimento:** la distribución y los costes constantes determinan qué estrategia gana para una entrada concreta.

Exhaustivo ofrece una referencia simple y fiable. ExhaustivoPoda resulta excelente cuando la separación horizontal descarta candidatos. DyV básico muestra que dividir no basta para garantizar una mejora asintótica. DyV mejorado aporta la garantía $\Theta(n \log n)$ al conservar el orden vertical y limitar geométricamente los candidatos.

La evidencia experimental confirma el modelo: la poda horizontal domina en el caso general probado, mientras que DyV mejorado es claramente superior en la distribución vertical desfavorable.

Referencias

1. Enunciado *Práctica 1 — Algoritmos Exhaustivos y Divide y Vencerás*, versión 24.10, Algorítmica y Modelos de Computación.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest y Clifford Stein. *Introduction to Algorithms*, 4.ª edición. MIT Press, 2022.
3. MDN — Web Workers API.
4. MDN — `performance.now()`.

5. Angular Documentation.

Referencia técnica

La documentación generada de tipos, funciones y módulos está disponible en la referencia TypeDoc.